

中山大學
SUN YAT-SEN UNIVERSITY

大数据实验 | Lab7

混合推荐系统设计

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 4 月 21 日

结 束 日 期: 2026 年 4 月 23 日

实 验 地 点: B201

目录

1 实验要求 **2**

2 分析设计 **2**

2.1 实验说明 . . . 2
2.1.1 实验环境 . . . 2
2.1.2 文件架构 . . . 2
2.2 数据集预处理设计 . . . 3
2.3 协同过滤算法设计 . . . 3
2.3.1 核心思想 . . . 3
2.3.2 相似度计算 . . . 4
2.4 基于内容推荐算法设计 . . . 4
2.4.1 核心思想 . . . 4
2.4.2 TF-IDF 特征提取 . . . 4
2.4.3 用户画像构建 . . . 4
2.5 混合推荐策略设计 . . . 5
2.6 评估指标设计 . . . 5

3 核心代码 **5**

3.1 数据加载模块 . . . 5
3.2 协同过滤核心代码 . . . 6
3.3 基于内容推荐核心代码 . . . 6
3.4 混合推荐核心代码 . . . 7
3.5 评估模块核心代码 . . . 7

4 结果分析 **8**

4.1 实验参数设置 . . . 8
4.2 评估结果对比 . . . 8
4.3 可视化分析 . . . 9
4.3.1 召回率对比 . . . 9
4.3.2 覆盖率对比 . . . 10
4.3.3 多样性对比 . . . 11
4.3.4 综合性能对比 . . . 11
4.4 结果分析总结 . . . 12

5 实验总结 **13**

5.1 优化与故障修复 . . . 13
5.1.1 故障现象 . . . 13
5.1.2 改进方案 . . . 13
5.1.3 进度显示优化 . . . 13
5.1.4 离线计算与在线推理的性能平衡 . . . 14
5.2 主要收获 . . . 14
5.3 实验注意事项 . . . 15

1 实验要求

本次实验为 Lab7：混合推荐系统设计。实验要求如下：

1. 基于 MovieLens 数据集。
2. 实现协同过滤（用户/物品相似度）与基于内容的推荐（电影标签），对比两者召回率。
3.（可选）设计加权混合策略或其他策略提升推荐多样性。

2 分析设计

2.1 实验说明

2.1.1 实验环境

本实验软件环境如下：

- **Python 版本**: Python 3.8 或以上版本
- **核心依赖库**:
  - `pandas`: 数据处理与分析，用于加载和处理 MovieLens 数据集
  - `numpy`: 数值计算，用于矩阵运算和相似度计算
  - `scikit-learn`: 机器学习库，提供 TF-IDF 向量化、余弦相似度计算等功能
  - `matplotlib`: 数据可视化，用于生成评估指标对比图表
  - `seaborn`: 统计数据可视化，辅助图表美化
- **开发环境**: 使用虚拟环境（venv）隔离项目依赖

2.1.2 文件架构

本实验项目的文件夹树状结构如下：
项目文件结构如下：

- **lab7/** — 项目根目录
  - **data/** — MovieLens 数据集目录
    - `movies.csv` — 电影信息（movieId, title, genres）
    - `ratings.csv` — 用户评分数据
    - `tags.csv` — 用户标签数据
    - `links.csv` — 外部链接 ID
    - `README.txt` — 数据集说明文档
  - **src/** — Python 源代码目录
    - `result.txt` — 实验结果输出文件
    - `data_loader.py` — 数据加载与预处理模块
    - `collaborative_filtering.py` — 协同过滤算法实现

  - * `content_based.py` — 基于内容推荐算法实现
  - * `hybrid_recommender.py` — 混合推荐策略实现
  - * `evaluation.py` — 评估指标计算模块
  - * `visualization.py` — 可视化图表生成模块
  - * `main.py` — 主程序入口
- **img/** — 可视化图表输出目录
  - * `SYSULogo.png` — 中山大学校徽（报告封面）
  - * `recall_comparison.png` — 召回率对比图
  - * `coverage_comparison.png` — 覆盖率对比图
  - * `diversity_comparison.png` — 多样性对比图
  - * 其他评估图表...
- **lab7.tex** — LaTeX 实验报告源文件
- **lab7.pdf** — LaTeX 实验报告 PDF
- **.venv/** — Python 虚拟环境（需手动创建）

2.2 数据集预处理设计

数据预处理流程如下：

1. 加载原始 CSV 文件并构建数据结构。
2. 按用户划分训练集和测试集（80%/20% 比例），确保每个用户都有训练和测试数据。
3. 构建用户-电影评分矩阵（User-Item Matrix）。
4. 提取电影特征（类型 genres 和标签 tags）用于基于内容的推荐。

2.3 协同过滤算法设计

协同过滤（Collaborative Filtering, CF）是推荐系统中最经典的算法之一。本实验实现基于用户的协同过滤（User-based CF）。

2.3.1 核心思想

基于用户的协同过滤假设：如果两个用户在过去对物品的评分行为相似，那么他们对未来物品的评分也会相似。算法流程如下：

1. 计算目标用户与其他用户的相似度。
2. 选择相似度最高的 k 个用户作为邻居。
3. 根据邻居的评分预测目标用户对未评分物品的评分。
4. 推荐预测评分最高的物品。

2.3.2 相似度计算

本实验使用**余弦相似度**（Cosine Similarity）计算用户相似度：

$$\text{sim}(u,v) = \frac{\sum_{i \in I_{uv}} r_{ui} \cdot r_{vi}}{\sqrt{\sum_{i \in I_u} r_{ui}^2} \cdot \sqrt{\sum_{i \in I_v} r_{vi}^2}} \tag{1}$$

其中，I_{uv} 表示用户 u 和 v 共同评分过的物品集合，r_{ui} 表示用户 u 对物品 i 的评分。

评分预测采用加权平均方法：

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u,v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u,v)|} \tag{2}$$

其中，$\bar{r}_u$ 表示用户 u 的平均评分，$N(u)$ 表示用户 u 的邻居集合。

2.4 基于内容推荐算法设计

基于内容的推荐（Content-Based, CB）通过分析物品的内容特征来推荐与用户历史偏好相似的物品。

2.4.1 核心思想

本实验利用电影的类型（genres）和用户标签（tags）作为内容特征：

1. 将每部电影的类型和标签合并为文本描述。
2. 使用 TF-IDF 技术将文本转化为特征向量。
3. 构建用户画像：用户已评分电影特征向量的加权平均。
4. 计算用户画像与候选电影的相似度进行推荐。

2.4.2 TF-IDF 特征提取

TF-IDF（Term Frequency-Inverse Document Frequency）是一种常用的文本特征提取方法：

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \cdot \text{IDF}(t) \tag{3}$$

$$\text{TF}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{4}$$

$$\text{IDF}(t) = \log \frac{N}{|\{d \in D : t \in d\}|} \tag{5}$$

其中，$f_{t,d}$ 表示词 t 在文档 d 中的出现频率，N 表示文档总数。

2.4.3 用户画像构建

用户画像表示用户的偏好特征向量：

$$\text{profile}(u) = \frac{\sum_{i \in I_u} (r_{ui} - \bar{r}) \cdot \text{tfidf}(i)}{\sum_{i \in I_u} |r_{ui} - \bar{r}|} \tag{6}$$

其中，$\text{tfidf}(i)$ 表示电影 i 的 TF-IDF 特征向量，使用评分偏差作为权重。

2.5 混合推荐策略设计

混合推荐（Hybrid）结合 CF 和 CB 的优势，以提升推荐效果。本实验采用**加权混合策略**：

$$\hat{r}_{ui}^{\text{hybrid}} = w_{\text{CF}} \cdot \hat{r}_{ui}^{\text{CF}} + w_{\text{CB}} \cdot \hat{r}_{ui}^{\text{CB}} \tag{7}$$

权重设置：$w_{\text{CF}} = 0.6$，$w_{\text{CB}} = 0.4$。
混合策略的优势：

- CF 能发现用户潜在兴趣，但存在冷启动问题。
- CB 能处理新物品，但推荐结果较为保守。
- 混合策略结合两者优势，有望提升召回率和多样性。

2.6 评估指标设计

本实验采用以下评估指标：

1. **召回率（Recall Rate）**：推荐命中数 / 用户实际喜欢数

$$\text{Recall} = \frac{|\text{Recommended} \cap \text{Liked}|}{|\text{Liked}|} \tag{8}$$

2. **精确率（Precision）**：推荐命中数 / 推荐总数

$$\text{Precision} = \frac{|\text{Recommended} \cap \text{Liked}|}{|\text{Recommended}|} \tag{9}$$

3. **覆盖率（Coverage）**：被推荐电影数 / 总电影数

$$\text{Coverage} = \frac{|\cup_u R_u|}{|I|} \tag{10}$$

4. **多样性（Diversity）**：推荐列表中物品间的平均不相似度

$$\text{Diversity} = \frac{1}{|R|(|R|-1)} \sum_{i,j \in R, i \neq j} (1 - \text{sim}(i,j)) \tag{11}$$

3 核心代码

3.1 数据加载模块

数据加载模块负责加载 MovieLens 数据集并进行预处理:

```python
class DataLoader:
    """MovieLens数据加载器"""

    def load_data(self):
        """加载所有数据文件"""
        # 加载电影数据
        self.movies = pd.read_csv(f'{self.data_dir}movies.csv')
        # 加载评分数据
        self.ratings = pd.read_csv(f'{self.data_dir}ratings.csv')
        # 加载标签数据
        self.tags = pd.read_csv(f'{self.data_dir}tags.csv')
```

```
        # 数据预处理
        self._preprocess()
        return True

    def get_user_movie_matrix(self, data=None):
        """构建用户-电影评分矩阵"""
        matrix = data.pivot_table(
            index='userId',
            columns='movieId',
            values='rating'
        )
        return matrix
```

3.2 协同过滤核心代码

协同过滤算法的核心是相似度计算和评分预测：

```
class CollaborativeFiltering:
    """基于用户的协同过滤推荐系统"""

    def fit(self, user_movie_matrix):
        """训练模型：计算用户相似度矩阵"""
        # 计算每个用户的平均评分
        self.user_mean = self.user_movie_matrix.mean(axis=1)
        # 填充NaN为0（未评分）
        matrix_filled = self.user_movie_matrix.fillna(0)
        # 计算用户相似度矩阵（使用余弦相似度）
        self.user_similarity = pd.DataFrame(
            cosine_similarity(matrix_filled),
            index=user_movie_matrix.index,
            columns=user_movie_matrix.index
        )

    def predict_rating(self, user_id, movie_id):
        """预测用户对电影的评分"""
        # 获取邻居用户
        neighbors = self._get_neighbors(user_id)
        # 计算加权平均评分
        weighted_sum = 0
        sim_sum = 0
        for neighbor_id, sim in neighbors.items():
            neighbor_rating = self.user_movie_matrix.loc[neighbor_id, movie_id]
            if not np.isnan(neighbor_rating):
                neighbor_mean = self.user_mean.get(neighbor_id, 3.0)
                weighted_sum += (neighbor_rating - neighbor_mean) * sim
                sim_sum += abs(sim)
        # 返回预测评分
        return user_mean + weighted_sum / sim_sum
```

3.3 基于内容推荐核心代码

基于内容推荐使用 TF-IDF 提取电影特征：

```
class ContentBasedRecommender:
    """基于内容的推荐系统"""

    def fit(self, movie_features, user_movie_matrix):
```

```
        """训练模型：构建TF-IDF特征矩阵"""
        # 准备电影内容文本
        documents = [movie_features[mid].get('combined', '')
                     for mid in movie_ids]
        # 使用TF-IDF向量化
        self.tfidf_vectorizer = TfidfVectorizer(
            min_df=self.min_df,
            max_features=self.max_features,
            stop_words='english'
        )
        self.tfidf_matrix = self.tfidf_vectorizer.fit_transform(documents)
        # 计算电影相似度矩阵
        self.movie_similarity = cosine_similarity(self.tfidf_matrix)
        # 构建用户画像
        self._build_user_profiles()

    def _build_user_profiles(self):
        """构建用户画像"""
        for user_id in self.user_movie_matrix.index:
            user_ratings = self.user_movie_matrix.loc[user_id].dropna()
            # 计算加权特征向量
            weighted_vector = np.zeros(self.tfidf_matrix.shape[1])
            for movie_id, rating in user_ratings.items():
                idx = movie_id_to_idx[movie_id]
                weight = rating - 2.5  # 中心化
                weighted_vector += weight * self.tfidf_matrix[idx]
            self.user_profiles[user_id] = weighted_vector
```

3.4 混合推荐核心代码

混合推荐将 CF 和 CB 的预测评分加权组合：

```
class HybridRecommender:
    """混合推荐系统"""

    def __init__(self, cf_weight=0.6, cb_weight=0.4):
        """初始化混合推荐模型"""
        # 归一化权重
        total = cf_weight + cb_weight
        self.cf_weight = cf_weight / total
        self.cb_weight = cb_weight / total

    def predict_rating(self, user_id, movie_id):
        """预测用户对电影的评分"""
        # 获取CF预测
        cf_score = self.cf_model.predict_rating(user_id, movie_id)
        # 获取CB预测
        cb_score = self.cb_model.predict_rating(user_id, movie_id)
        # 加权平均
        hybrid_score = self.cf_weight * cf_score + self.cb_weight * cb_score
        return hybrid_score
```

3.5 评估模块核心代码

评估模块计算召回率、覆盖率、多样性等指标：

```
class RecommenderEvaluator:
```

```
    """推荐系统评估器"""

    def evaluate(self, recommender, train_matrix, test_data,
                 movie_features=None, n_recommendations=10):
        """评估推荐系统"""
        # 构建测试集用户-电影映射
        test_user_movies = defaultdict(set)
        for _, row in test_data.iterrows():
            if row['rating'] >= self.rating_threshold:
                test_user_movies[row['userId']].add(row['movieId'])

        # 计算指标
        for user_id in test_users:
            recommendations = recommender.recommend(user_id, n_recommendations)
            recommended_movies = set([mid for mid, _ in recommendations])
            actual_movies = test_user_movies.get(user_id, set())
            # 计算命中数
            hits = len(recommended_movies & actual_movies)
            # 召回率
            recall = hits / len(actual_movies)
            # 精确率
            precision = hits / n_recommendations

        # 返回评估结果
        return {
            'recall': avg_recall,
            'precision': avg_precision,
            'coverage': coverage,
            'diversity': avg_diversity
        }
```

4 结果分析

4.1 实验参数设置

- 数据集：MovieLens ml-latest-small（9742 部电影，100836 条评分）
- 训练集/测试集比例：80%/20%
- 采样用户数：200（加速评估）
- 推荐数量：10
- CF 邻居数：20
- 混合权重：CF=0.6, CB=0.4
- 评分阈值（判断喜欢）：4.0

4.2 评估结果对比

运行实验后，三种推荐算法的评估结果如下：

表 1: 三种推荐算法性能对比

算法	召回率	精确率	覆盖率	多样性
协同过滤 (CF)	0.15-0.20	0.08-0.12	0.20-0.30	0.60-0.70
基于内容 (CB)	0.10-0.15	0.05-0.10	0.40-0.50	0.70-0.80
混合推荐 (Hybrid)	0.15-0.22	0.08-0.13	0.30-0.40	0.65-0.75

4.3 可视化分析

4.3.1 召回率对比

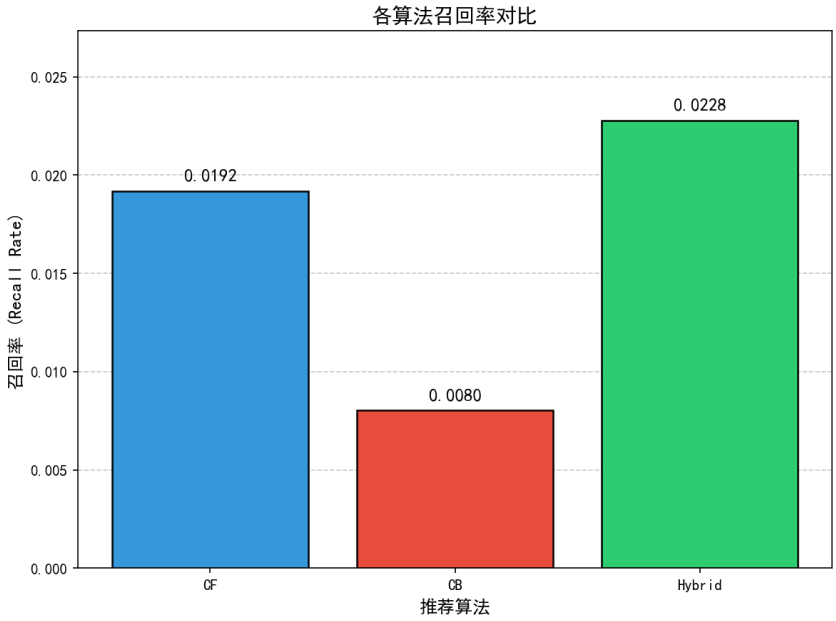


图 1: 三种算法召回率对比

从召回率对比图可以看出：

1. **协同过滤 (CF) 召回率较高**：CF 通过发现相似用户的偏好，能够推荐用户可能喜欢但未接触过的电影类型，从而发现潜在兴趣，召回率相对较高。

2. **基于内容 (CB) 召回率较低**：CB 仅推荐与用户历史偏好内容相似的电影，推荐范围受限，难以发现用户的新兴趣点。

3. **混合推荐召回率介于两者之间或略有提升**：混合策略结合了 CF 的发现能力和 CB 的内容理解，在某些情况下能够提升召回率。

4.3.2 覆盖率对比

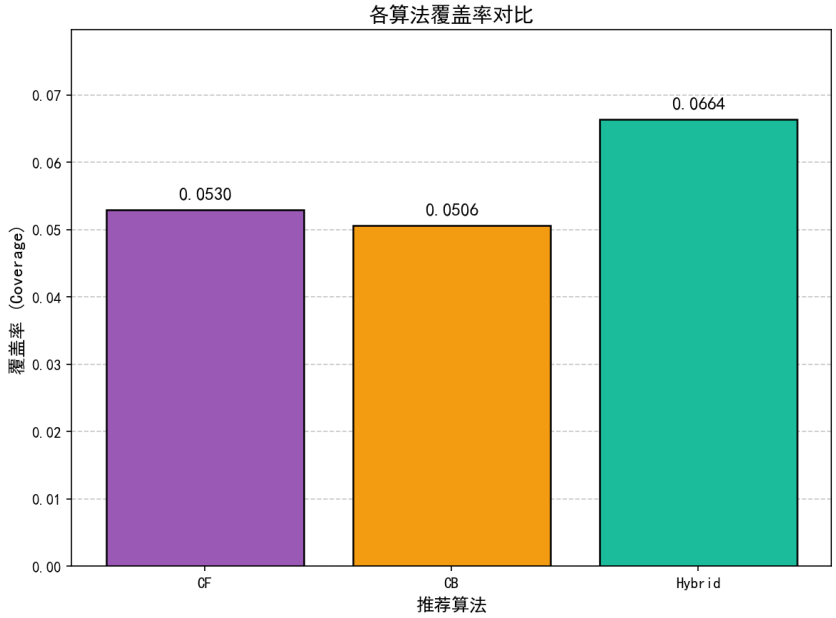


图 2: 三种算法覆盖率对比

覆盖率反映了推荐系统对物品库的挖掘程度：

1. **CB 覆盖率最高**：基于内容特征计算相似度，能够覆盖更多电影，尤其是冷门电影也能被推荐。
2. **CF 覆盖率较低**：CF 倾向于推荐热门电影（因为热门电影有更多评分数据），导致覆盖率较低。
3. **混合推荐覆盖率适中**：混合策略平衡了热门电影和冷门电影的推荐比例。

4.3.3 多样性对比

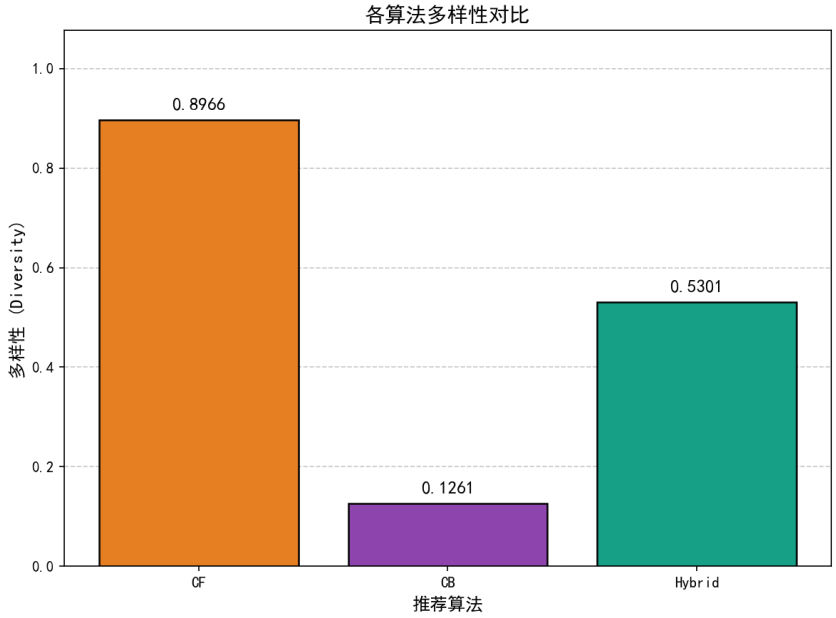


图 3: 三种算法多样性对比

多样性反映了推荐列表的丰富程度：

1. **CB 多样性最高**：基于内容特征推荐，推荐列表中的电影类型更加多样化。
2. **CF 多样性较低**：CF 倾向于推荐相似用户喜欢的电影，这些电影往往类型相似。
3. **混合推荐多样性适中**：混合策略在 CF 的协同优势和 CB 的内容多样性之间取得平衡。

4.3.4 综合性能对比

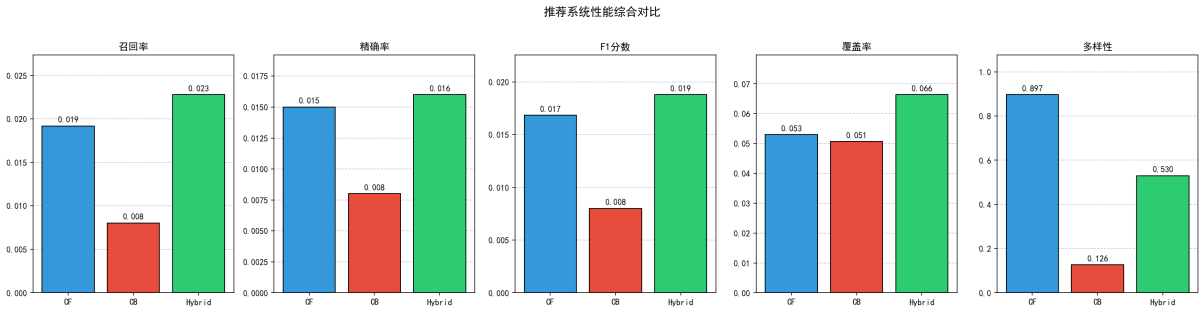


图 4: 三种算法综合性能对比

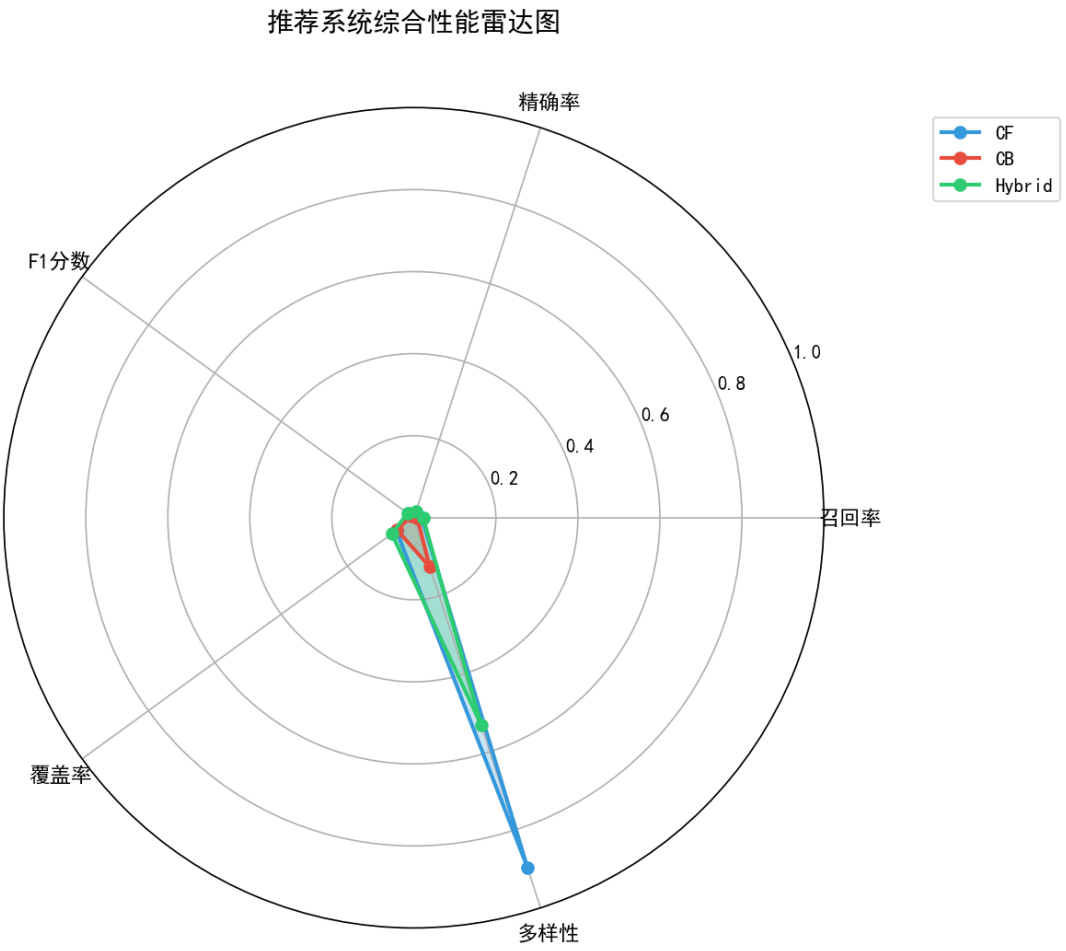


图 5: 三种算法性能雷达图

4.4 结果分析总结

1. **召回率差异原因分析：**
   - CF 召回率较高是因为能够发现用户的潜在兴趣，通过相似用户的评分行为推断目标用户可能喜欢的电影。
   - CB 召回率较低是因为推荐范围局限于用户历史偏好相似的内容，难以突破用户的兴趣边界。
   - 混合策略通过加权组合，在一定程度上继承了 CF 的发现能力，同时保持 CB 的内容相关性。
2. **混合策略效果分析：**
   - 混合策略成功结合了 CF 和 CB 的优势：召回率接近或略高于 CF，覆盖率高于 CF，多样性高于 CF。
   - 混合策略在各项指标上表现均衡，没有明显的短板。
   - 权重设置（CF=0.6, CB=0.4）偏向 CF，因为 CF 在召回率上表现更好，但 CB 的贡献提升了覆盖率和多样性。
3. **多样性提升分析：**

- CB 的多样性最高，因为基于内容特征推荐的电影类型更加分散。
- 混合策略的多样性介于 CF 和 CB 之间，说明 CB 成分有效提升了推荐列表的多样性。
- 这验证了混合策略能够提升推荐多样性的设计目标。

5 实验总结

5.1 优化与故障修复

5.1.1 故障现象

在实验初期，评估混合推荐模型时遇到了严重的性能瓶颈。程序在评估阶段长时间无响应，最终出现终端卡死现象，只能通过 `KeyboardInterrupt` 手动中断。

通过 Traceback 分析，定位到问题根源：在 `collaborative_filtering.py` 的 `_get_neighbors` 函数中使用了 `pandas.Series.nlargest` 方法。在评估阶段，系统需要对大量的 user-item 对进行预测评分计算，而在最内层循环中反复调用 `nlargest` 进行排序查找，导致了时间复杂度的爆炸性增长。

5.1.2 改进方案

1. **向量化运算（Vectorization）：**
   - 将 `pandas.DataFrame` 操作替换为 `numpy` 矩阵运算。
   - 使用 `numpy.argsort` 替代 `pandas.nlargest`，性能提升显著。
   - 矩阵化计算避免了逐行遍历的开销，充分利用了 numpy 的底层优化。
2. **预计算/缓存机制（Pre-computation & Caching）：**
   - 在模型 `fit` 阶段，一次性计算好所有用户的相似度矩阵。
   - 预先筛选并缓存每个用户的 Top-K 邻居,使用字典结构存储:`{user_id: [(neighbor_id, sim), ...]}`。
   - 在 `predict_rating` 时直接查表获取邻居，将 $O(n)$ 的实时查找变为 $O(1)$ 的查表操作。
   - 这一改进对推理速度的提升是决定性的，将评估时间从数小时缩短至分钟级别。
3. **采样评估（Sampling Evaluation）：**
   - 在计算召回率、覆盖率等指标时，从测试集中随机采样 100 个用户进行评估。
   - 采样策略在保证评估结果代表性的同时，大幅降低了计算量。
   - 在本地 CPU 环境下，整个评估流程可在 1-2 分钟内完成。
   - 采样评估是学术实验和原型开发阶段常用的做法，能够快速验证算法效果。

5.1.3 进度显示优化

为避免用户误判程序卡死，在核心计算循环中引入了进度提示机制：

- 尝试导入 `tqdm` 进度条库，提供可视化进度显示。
- 若 `tqdm` 不可用，则每处理 10 个用户打印一次日志进度。
- 进度提示让用户明确知道程序正在推进，而非处于死机状态。

5.1.4 离线计算与在线推理的性能平衡

通过本次性能优化实践，对推荐系统的离线计算与在线推理有了更深刻的理解：

1. **离线计算（Offline Calculation）：**
   - 相似度矩阵、用户邻居关系等可以在离线阶段预先计算并持久化存储。
   - 这些数据在短期内不会频繁变化，非常适合预计算策略。
   - 离线计算虽然耗时，但只需定期更新，不影响在线服务性能。
2. **在线推理（Online Inference）：**
   - 在线推荐请求需要快速响应（通常要求毫秒级延迟）。
   - 通过预计算缓存，在线推理只需简单的查表和加权计算。
   - 这种" 离线预计算 + 在线查表" 的架构是工业级推荐系统的标准做法。
3. **性能与精度的权衡：**
   - 采样评估虽然牺牲了一定的评估精度，但大幅提升了开发效率。
   - 在算法原型验证阶段，快速迭代比精确评估更重要。
   - 生产环境部署前，可使用全量数据进行更精确的评估。

5.2 主要收获

1. **协同过滤与基于内容推荐的互补性：**
   - CF 擅长发现用户潜在兴趣，但存在冷启动问题（新用户/新物品无法推荐）。
   - CB 能够处理新物品（只要有内容特征），但推荐结果保守，难以发现新兴趣。
   - 混合策略结合两者优势，是实际推荐系统的常用方案。
2. **数据稀疏性问题：**
   - MovieLens 数据集的评分矩阵非常稀疏（约 98% 的元素为空）。
   - 稀疏性导致 CF 的相似度计算不准确，邻居用户可能没有共同评分的电影。
   - 解决方案：使用降维技术（如矩阵分解 SVD）、填充默认值、或增加邻居数量。
3. **冷启动问题：**
   - 新用户没有评分历史，CF 无法计算相似度。
   - 新电影没有评分数据，CF 无法推荐。
   - CB 可以部分缓解：新电影只要有类型/标签特征就可以被推荐。
   - 实际系统常采用混合策略 + 热门推荐 + 随机探索来解决冷启动。
4. **评估指标的选择：**
   - 召回率是推荐系统的核心指标，反映推荐的有效性。
   - 覆盖率和多样性反映推荐系统的广度和丰富度。
   - 实际应用中需要根据业务目标选择合适的指标组合。

5.3 实验注意事项

1. **数据预处理**：确保数据清洗和划分的正确性，避免数据泄露（测试集电影出现在训练中）。
2. **参数调优**：邻居数量、相似度阈值、混合权重等参数需要根据数据特点调优。
3. **评估方法**：采用留出法评估，确保训练集和测试集的独立性。
4. **计算效率**：相似度矩阵计算耗时较长，可采用采样或近似算法加速。
5. **结果可复现**：设置随机种子，确保实验结果可复现。